Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Artificial Intelligence
Agentic AI (AI465), Semester VII

Instructions

1. Save your file as LAB_No_Roll_No.doc (e.g. LAB_4_U22AI001.doc).
2. Paste a snapshot of each step followed along with the result(s).
3. Record your observations and comments after the results.

Experiment 4

Retrieval-Augmented Generation — Grounding an Agent in a Document Store

Aim

To build a retrieval-augmented generation pipeline — chunk a corpus, vectorise it into a store, retrieve the nearest chunks for a query and assemble a grounded prompt — and to compare its cited answers against an ungrounded baseline.

Theory

A language model carries its knowledge in its weights. That knowledge is fixed at training time, cannot be audited, and is produced with the same fluency whether the model has the fact or is inventing it. Retrieval-augmented generation moves the knowledge out of the weights and into a store the developer controls: the corpus is split into chunks, each chunk is turned into a vector, and at query time the chunks nearest the question are fetched and pasted into the prompt as evidence. The model's task narrows from recall to reading comprehension, the corpus can be updated without retraining, and because every retrieved chunk carries its source identifier, each answer can be traced back to the text that supports it.

This experiment implements the retrieval half completely and scripts the generation step, so the whole pipeline is visible without an API key or network access. The chunks are vectorised with TF-IDF and searched by cosine similarity; substituting dense sentence embeddings and a FAISS index replaces only the vectoriser and the search call, and leaves the chunking, context assembly and citation logic unchanged.

Note the two limitations visible in the sample run. First, top-*k* retrieval always returns *k* chunks, even for a question the corpus cannot answer, so the pipeline will ground an answer in whatever text happened to score highest; a similarity threshold is needed before the pipeline is allowed to answer at all (Exercise 1). Second, retrieval can only return what chunking has made retrievable — a chunk boundary that cuts an answer in half makes that answer unreachable at any *k* (Exercise 2). Note also that the pipeline here retrieves exactly once per query; the agentic variant, where the agent itself decides when to retrieve inside a reasoning loop, is taken up in a later experiment.

Table 1. Stages of the pipeline and the design choice each one carries.

Stage	What it produces	Design choice exercised here
Chunking	A list of text chunks, each tagged with its source document and position	Chunk size and overlap, in sentences
Vectorisation	One vector per chunk	TF-IDF over the corpus vocabulary, with stop words removed
Indexing	The vector store — the chunk matrix searched at query time	Exhaustive search over a dense matrix, in place of an approximate index
Retrieval	The top-*k* chunks with their similarity scores	Value of *k*, and the cosine similarity measure
Context assembly	A grounded prompt: instruction, cited chunks, question	Citation format and the instruction to use only the supplied context
Generation	The final answer with its source tag	Scripted stand-in for the model, so the retrieval is what is being measured

Algorithm

1. Build the index: split every document in the corpus into overlapping chunks, tag each chunk with its source document and position, vectorise all chunks together, and keep the resulting matrix as the vector store.
2. Answer a query: vectorise the query, score it by cosine similarity against every chunk, take the top-*k*, assemble a grounded prompt from those chunks and their source tags, and print the cited answer next to the answer the same question receives with no retrieval at all.

Procedure and Result

Paste the snapshots of the steps followed and the output produced by the pipeline below, including the number of chunks indexed, and for every query the chunks retrieved with their similarity scores, the grounded answer with its citation, and the ungrounded baseline answer to the same question.

Observations

Comment on what the similarity scores of the retrieved chunks tell you about how confident the retrieval step is, on how the grounded and baseline answers differ in both content and traceability, and on what happens to the retrieved set as *k* is increased.

Exercises

1. Add a similarity threshold so that the pipeline abstains — reporting that the answer is not in the corpus — whenever the best retrieved chunk scores below it. Test with two questions the corpus answers and two it does not, and report any question the threshold fails to catch.
2. Vary the chunk size and overlap over at least four settings, and for a fixed set of questions report the rank and similarity score of the chunk that actually contains the answer. State which setting you would ship and why.